

Documentation Technique – Authentification

Projet To Do & Co

Introduction

Cette documentation vise à guider les développeurs juniors dans la compréhension de l'implémentation de l'authentification. Elle couvre les principaux fichiers concernés par cette fonctionnalité.

1. Emplacement des fichiers

Configuration :

config/packages/security.yaml : Configuration des paramètres de sécurité.

.env.local : Configuration des variables d'environnement locales.

Classes :

src/Entity/User.php : Entité représentant l'utilisateur.

src/Controller/LoginController.php : Contrôleur de connexion.

src/EventListener/UserRedirectListener.php : Fichier personnalisé pour la redirection des utilisateurs déjà connecté.

Vues :

templates/base_home.html.twig : Template de base pour les pages du site.

templates/security/login.html.twig : Template pour la page de connexion.

templates/_partials/_flash_messages.html.twig : Template pour les messages flash.

2. Processus d'authentification

Cette section explique comment Symfony gère le processus d'authentification, couvrant la configuration du firewall, la création de formulaires de connexion, et le traitement des requêtes de connexion dans le contrôleur.

2.1 Configuration du Firewall

Le fichier de configuration principal pour la sécurité dans Symfony est security.yaml. Voici le code de ce fichier avec les principales configurations liées à l'authentification :

```

security:
    password_hashers:
        Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface: 'auto'

    providers:
        app_user_provider:
            entity:
                class: App\Entity\User
                property: username

    firewalls:
        dev:
            pattern: ^/(_(profiler|wdt)|css|images|js)/
            security: false
        main:
            lazy: true
            provider: app_user_provider
            form_login:
                login_path: login
                check_path: login
                default_target_path: /tasks/list
            logout:
                path: logout
                target: /home

    access_control:
        - { path: ^/user/list, roles: ROLE_ADMIN }

    role_hierarchy:
        ROLE_ADMIN: [ROLE_USER]

when@test:
    security:
        password_hashers:
            Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface:
                algorithm: auto
                cost: 4
                time_cost: 3
                memory_cost: 10

```

Dans cette configuration, le firewall principal (main) est défini avec un fournisseur d'utilisateurs (app_user_provider) basé sur l'entité User. La méthode de hachage automatique des mots de passe est configurée, et le formulaire de connexion est défini avec des chemins spécifiques.

2.2 Création de formulaires de connexion

Le contrôleur LoginController.php gère le processus de connexion. Voici le code :

```

<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;
use Symfony\Component\Security\Http\Authentication\AuthenticationUtils;

class LoginController extends AbstractController
{
    #[Route('/login', name: 'login')]
    public function loginAction(AuthenticationUtils $authenticationUtils): Response
    {
        if ($this->isGranted('IS_AUTHENTICATED_FULLY')) {
            return $this->redirectToRoute('app_home');
        }

        $error = $authenticationUtils->getLastAuthenticationError();
        $lastUsername = $authenticationUtils->getLastUsername();

        return $this->render('security/login.html.twig', [
            'last_username' => $lastUsername,
            'error' => $error,
        ]);
    }

    #[Route('/logout', name: 'logout')]
    public function logout()
    {
        // Nothing to do here...
    }
}

```

Ce contrôleur utilise le service AuthenticationUtils pour récupérer les erreurs d'authentification et le dernier nom d'utilisateur saisi. Il rend ensuite le template Twig correspondant.

2.3 Traitement des requêtes de connexion dans le contrôleur

Le formulaire de connexion est rendu dans le template Twig login.html.twig. Ce fichier utilise les données fournies par le contrôleur, et le formulaire POST est géré par le même contrôleur.

Le processus d'authentification est géré automatiquement par Symfony grâce à la configuration du firewall, et une fois l'authentification réussie, l'utilisateur est redirigé vers la page par défaut spécifiée dans la configuration (/tasks/list dans ce cas).

2.4 Redirection des utilisateurs

Le fichier UserRedirectListener.php est un listener personnalisé pour la redirection des utilisateurs. Son objectif est de rendre la page login.html.twig inaccessible si un utilisateur est déjà connecté.

Celui-ci sera redirigé vers une page spécifique, dans ce cas, la page d'accueil de l'application ([home.html.twig](#)). Voici le code du fichier UserRedirectListener.php :

```
<?php

namespace App\EntityListener;

use Symfony\Component\HttpFoundation\Event\RequestEvent;
use Symfony\Component\Routing\Generator\UrlGeneratorInterface;
use Symfony\Component\Security\Core\Authorization\AuthorizationCheckerInterface;

class UserRedirectListener
{
    private $authorizationChecker;
    private $urlGenerator;

    public function __construct(AuthorizationCheckerInterface $authorizationChecker, UrlGeneratorInterface $urlGenerator)
    {
        $this->authorizationChecker = $authorizationChecker;
        $this->urlGenerator = $urlGenerator;
    }

    public function onKernelRequest(RequestEvent $event)
    {
        $request = $event->getRequest();
        $route = $request->attributes->get('_route');

        if ($route === 'login' && $this->authorizationChecker->isGranted('IS_AUTHENTICATED_FULLY')) {
            // $response = new RedirectResponse($this->urlGenerator->generate('app_home'));
            // $event->setResponse($response);
        }
    }
}
```

Ce listener est correctement enregistré dans la configuration de services.yaml :

```
App\EntityListener\UserRedirectListener:
    arguments:
        $authorizationChecker: '@security.authorization_checker'
        $urlGenerator: '@router'
    tags:
        - { name: 'kernel.event_listener', event: 'kernel.request', method: 'onKernelRequest' }
```

3. Stockage des utilisateurs

Dans le contexte du projet Symfony ToDo & Co, les données des utilisateurs sont stockées dans une base de données, comme indiqué dans la configuration du composant de sécurité dans le fichier security.yaml.

3.1 Configuration de l'entité User

L'entité User est la représentation des utilisateurs dans l'application Symfony. Elle est définie dans le fichier src/Entity/User.php. Voici un extrait du code :

```
<?php

namespace App\Entity;

use App\Repository\UserRepository;
use Doctrine\ORM\Mapping as ORM;
use Symfony\Bridge\Doctrine\Validator\Constraints\UniqueEntity;
use Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface;
use Symfony\Component\Security\Core\User\UserInterface;
use Symfony\Component\Validator\Constraints as Assert;

#[ORM\EntityListeners(['App\EntityListener\UserListener'])]
#[UniqueEntity(fields: ['email'], message: 'This email is already used !')]
#[UniqueEntity(fields: ['username'], message: 'There is already an account with this username !')]
#[ORM\Entity(repositoryClass: UserRepository::class)]
class User implements UserInterface, PasswordAuthenticatedUserInterface
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column(type: 'integer')]
    private ?int $id = null;

    #[ORM\Column(type: 'string', length: 25)]
    #[Assert\NotBlank(message: 'Please enter a username !')]
    private ?string $username = null;

    #[ORM\Column(type: 'string', length: 60)]
    #[Assert\NotBlank(message: 'Please enter an email address !')]
    #[Assert\Email(message: 'The email is not a valid email address.')]
    private ?string $email = null;

    #[ORM\Column(type: 'json')]
    private $roles = [];

    #[Assert\NotBlank(message: 'Please enter a password !')]
    #[Assert\Regex(
        pattern: '/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$/',
        message: 'Your password must contain at least 8 characters, one lowercase letter, one uppercase letter, one number, and one special character.'
    )]
    private $plainPassword;

    #[ORM\Column(type: 'string', length: 255)]
    private ?string $password;
}
```

Dans cette classe, l'annotation `#[ORM\Entity(repositoryClass: UserRepository::class)]` indique que l'entité est mappée sur la base de données et utilise la classe `UserRepository` pour les opérations de base de données.

La colonne `roles` stocke les rôles attribués à chaque utilisateur. Les données de cette entité, y compris les mots de passe, sont persistées en base de données.

3.2 Configuration de la base de données

La configuration de la base de données se trouve généralement dans le fichier `.env.local`. Voici un extrait de fichier :

```
DATABASE_URL="mysql://root:"@127.0.0.1:3306/todoco?serverVersion=11.2.2-MariaDB"
```

Cela indique que l'application utilise une base de données MySQL/MariaDB avec le nom de la base de données `todoco`. Les identifiants de connexion tels que le nom d'utilisateur (`root`), le mot de passe (vide dans ce cas), l'hôte (`127.0.0.1`), et le port (`3306`) sont également spécifiés.

4. Informations additionnelles

Dans le cadre de l'authentification dans le projet `Symfony ToDo & Co`, plusieurs aspects spécifiques méritent une attention particulière.

4.1 Gestion des rôles

La gestion des rôles est essentielle pour définir les permissions d'accès des utilisateurs. Dans l'entité User définie dans le fichier src/Entity/User.php, la méthode getRoles() est utilisée pour attribuer le rôle ROLE_USER à chaque utilisateur par défaut. Voici un extrait de la méthode :

```
public function __construct()
{
    $this->roles = ['ROLE_USER'];
}
```

```
public function getRoles(): array
{
    return array_unique($this->roles);
}
```

De plus, dans le fichier security.yaml, la configuration access control définit des règles d'accès en fonction des rôles. Par exemple, l'accès à la liste des utilisateurs est limité au rôle ROLE_ADMIN. Voici un extrait de cette configuration :

```
access_control:
    - { path: ^/user/list, roles: ROLE_ADMIN }
```

Cette configuration garantit que seuls les utilisateurs avec le rôle ROLE_ADMIN peuvent accéder à la liste des utilisateurs.

4.2 Personnalisation des messages d'erreur

La personnalisation des messages d'erreur est gérée dans la vue du formulaire de connexion (security/login.html.twig). Les messages d'erreur spécifiques à l'authentification sont affichés en fonction des erreurs renvoyées par le composant de sécurité.

```
{% if error %}
    <div class="alert alert-danger" role="alert">{{ error.messageKey|trans(error.messageData, 'security') }}</div>
{% endif %}
```

Cette construction Twig affiche un message d'erreur en cas d'échec de l'authentification, fournissant ainsi des informations claires et conviviales aux utilisateurs.